WEB SECURITY

Web Basics

Consider an analogy of a web application as a planet. Astronauts travel to the planet to explore its surface, similar to how someone uses a web browser to explore or browse a web application. Although we only see the surface of a planet, a lot is going on under the surface. You can imagine the whole planet as a web server with many things going on under the surface of the web server, yet all we can usually see is the surface of web pages or apps. We will now explore the various components that make up a web application.

Front End

The **Front End** can be considered similar to the surface of the planet, the parts that an astronaut can see and interact with based on the laws of nature. A web application would have a user interact with it and use a number of technologies such as HTML, CSS, and JavaScript to do this.

HTML (Hypertext Markup Language) is a foundational aspect of web applications. It is a set of instructions or code that instructs a web browser on what to display and how to display it. It could be compared to simple organisms living on the planet; these organisms have DNA, which is the instructions for how simple organisms are put together.

CSS (Cascading Style Sheets) in web applications describes a standard appearance, such as certain colours, types of text, and layouts. Continuing the analogy with DNA, these could be compared to the parts of DNA that describe the colour, shape, size, and texture of the simple organism.

JS (JavaScript) is part of a web application front end that enables more complex activity in the web browser. Whereas HTML can be considered a simple set of instructions on what to display, JavaScript is a more advanced set of instructions that allows choices and decisions to be made on what to display. In the planet analogy,

JavaScript can be considered the brain of an advanced organism, which allows decisions to be made based on what and how something interacts with it.

Back End

The **Back End** of a web application is things you don't see within a web browser but are important for the web application to work. On a planet, these are the non-visual things: the structures that keep a building standing, the air, and the gravity that keeps feet on the ground.

A **Database** is where information can be stored, modified, and retrieved. A web application may want to store and retrieve information about a visitor's preferences on what to show or not; this would be stored in a database. A planet may have more advanced inhabitants who store information about locations in maps, write notes in a diary or put books in a library and files in a filing cabinet.

There are many other **Infrastructure** components underpinning Web Applications, such as web servers, application servers, storage, various networking devices, and other software that support the web application. On a planet, these are the roads that are present, the cars that run on those roads, the fuel that powers the cars.

WAF (Web Application Firewall) is an optional component for web applications. It helps filter out dangerous requests away from the Web Server and provides an element of protection. This could be considered similar to how a planet's atmosphere can protect inhabitants from harmful UV rays.

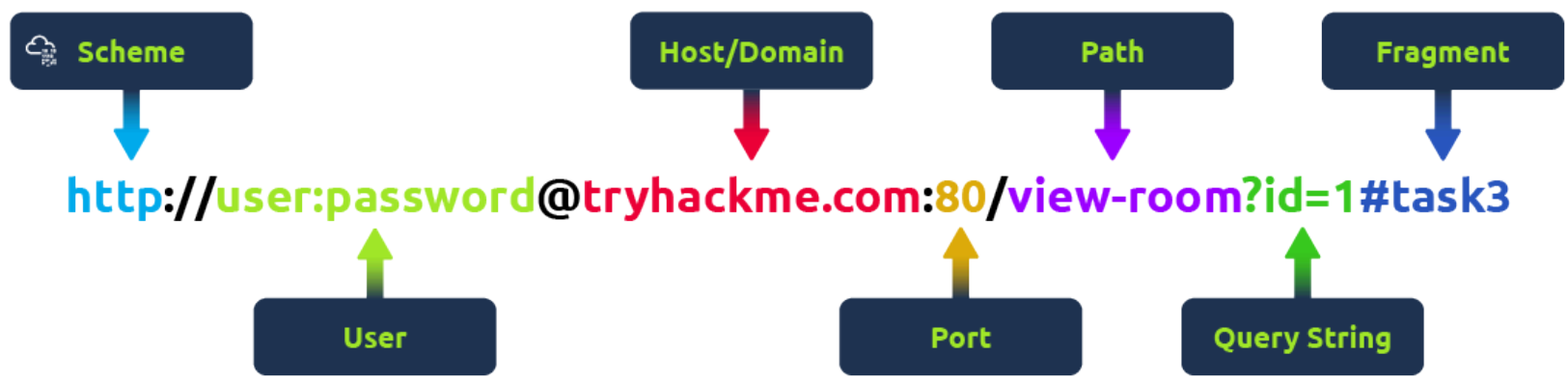
Summary

There are many components involved in delivering a web application. Front End components like HTML, CSS, and JavaScript focus on the experience inside the browser. Back End components such as the Web Server, Database, or WAF are the engine under the surface that enable the web application to function. This simple introduction will be built upon in the upcoming tasks.

Uniform Resource Locator

A Uniform Resource Locator (URL) is a web address that lets you access all kinds of online content—whether it's a webpage, a video, a photo, or other media. It guides your browser to the right place on the Internet.

Anatomy of a URL



Think of a URL as being made up of several parts, each playing a different role in helping you find the right resource. Understanding how these parts fit together is important for browsing the web, developing web applications, and even troubleshooting problems.

Here's a breakdown of the key components:

Scheme

The **scheme** is the protocol used to access the website. The most common are **HTTP** (HyperText Transfer Protocol) and **HTTPS** (Hypertext Transfer Protocol Secure). HTTPS is more secure because it encrypts the connection, which is why browsers and cyber security experts recommend it. Websites often enforce HTTPS for added protection.

User

Some URLs can include a user's login details (usually a username) for sites that require authentication. This happens mostly in URLs that need credentials to access certain resources. However, it's rare nowadays because putting login details in the URL isn't very safe—it can expose sensitive information, which is a security risk.

Host/Domain

The **host** or **domain** is the most important part of the URL because it tells you which website you're accessing. Every domain name has to be unique and is registered through domain registrars. From a security standpoint, look for domain names that appear almost like real ones but have small differences (this is called **typosquatting**). These fake domains are often used in phishing attacks to trick people into giving up sensitive info.

Port

The **port number** helps direct your browser to the right service on the web server. It's like telling the server which doorway to use for communication. Port numbers range from 1 to 65,535, but the most common are **80** for HTTP and **443** for HTTPS.

Path

The **path** points to the specific file or page on the server that you're trying to access. It's like a roadmap that shows the browser where to go. Websites need to secure these paths to make sure only authorised users can access sensitive resources.

Query String

The **query string** is the part of the URL that starts with a question mark (?). It's often used for things like search terms or form inputs. Since users can modify these query strings, it's important to handle them securely to prevent attacks like **injections**, where malicious code could be added.

Fragment

The **fragment** starts with a hash symbol (#) and helps point to a specific section of a webpage—like jumping directly to a particular heading or table. Users can modify this too, so like with query strings, it's important to check and clean up any data here to avoid issues like injection attacks.

Http Messages

HTTP messages are packets of data exchanged between a user (the client) and the web server. These messages are very important for understanding how web applications work because they show how users' requests and the server's responses are communicated.

Imagine an example of an HTTP Request and an HTTP Response, where you can see key parts like the method, URL, headers, and status codes. These are what make the client-server interaction possible.

There are two types of HTTP messages:

- **HTTP Requests**: Sent by the user to trigger actions on the web application.
- **HTTP Responses**: Sent by the server in response to the user's request.

HTTP Request

```
POST /login HTTP/1.1
Host: tryhackme.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 43

username=aleksandra&password=securepassword
```

HTTP Response

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 56
Date: Wed, 29 Aug 2024 12:00:00 GMT

{
  "message": "Login successful!",
  "status": "success"
}
```

Each message follows a specific format that helps both the user and the server communicate smoothly.

Start Line

The start line is like the introduction of the message. It tells you what kind of message is being sent—whether it's a request from the user or a response from the server. This line also gives important details about how the message should be handled.

Headers

Headers are made up of key-value pairs that provide extra information about the HTTP message. They give instructions to both the client and the server handling the request or response. These headers cover all sorts of things, like security, content types, and more, making sure everything goes smoothly in the communication.

Empty Line

The empty line is a little divider that separates the header from the body. It's essential because it shows where the headers stop and where the actual content of the message begins. Without this empty line, the message might get messed up, and the client or server could misinterpret it, causing errors.

Body

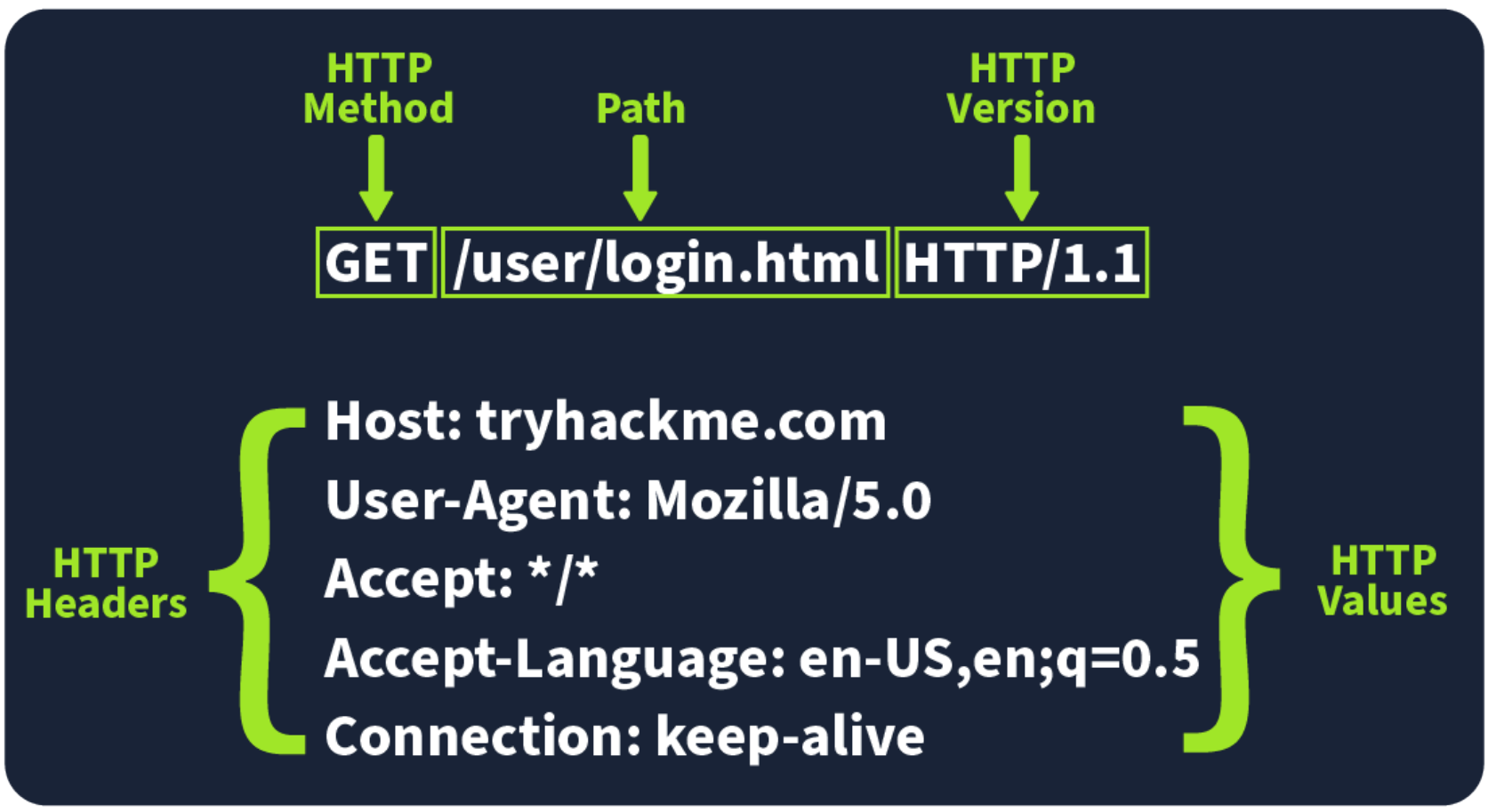
The body is where the actual data is stored. In a request, the body might include data the user wants to send to the server (like form data). In a response, it's where the server puts the content that the user requested (like a webpage or API data).

Why Understanding HTTP Messages Matters

- These messages are the foundation of how web applications communicate. If they're structured properly, everything works smoothly.
- Knowing how they work will help you diagnose issues in web communication, which means better performance and reliability for your web application.
- It's also crucial for security. Understanding HTTP messages helps you implement strong security measures to protect data during transmission

HTTP Request

An **HTTP request** is what a user sends to a web server to interact with a web application and make something happen. Since these requests are often the first point of contact between the user and the web server, knowing how they work is super important—especially if you're into cyber security.



Imagine an HTTP request showing the key parts like the method (e.g., GET or POST), path (e.g., /login), and version (e.g., HTTP/1.1). These elements make up the basics of how a client (user) communicates with a server.

Request Line

The **request line** (or start line) is the first part of an HTTP request and tells the server what kind of request it's dealing with. It has three main parts: the **HTTP method**, the **URL path**, and the **HTTP version**.

Example: `METHOD /path HTTP/version`

HTTP Methods

The **HTTP method** tells the server what action the user wants to perform on the resource identified by the URL path. Here are some of the most common methods and their possible security issue:

GETUsed to **fetch** data from the server without making any changes. Reminder! Make sure you're only exposing data the user is allowed to see. Avoid putting sensitive info like tokens or passwords in GET requests since they can show up as plaintext.

POSTSends data to the server, usually to create or update something. Reminder! Always validate and clean the input to avoid attacks like SQL injection or XSS.

PUTReplaces or **updates** something on the server. Reminder! Make sure the user is authorised to make changes before accepting the request.

DELETERemoves something from the server. Reminder! Just like with PUT, make sure only authorised users can delete resources.

Besides these common methods, there are a few others used in specific cases:

PATCHUpdates part of a resource. It's useful for making small changes without replacing the whole thing, but always validate the data to avoid inconsistencies.

HEADWorks like GET but only retrieves headers, not the full content. It's handy for checking metadata without downloading the full response.

OPTIONSTells you what methods are available for a specific resource, helping clients understand what they can do with the server.

TRACESimilar to OPTIONS, it shows which methods are allowed, often for debugging. Many servers disable it for security reasons.

CONNECTUsed to create a secure connection, like for HTTPS. It's not as common but is critical for encrypted communication.

Each of these methods has its own set of security rules. For example, PATCH requests should be validated to avoid inconsistencies, and OPTIONS and TRACE should be turned off if not needed to avoid possible security risks.

URL Path

The **URL path** tells the server where to find the resource the user is asking for. For instance, in the URL `https://tryhackme.com/api/users/123`, the path `/api/users/123` identifies a specific user.

Attackers often try to manipulate the URL path to exploit vulnerabilities, so it's crucial to:

- Validate the URL path to prevent unauthorised access
- Sanitise the path to avoid injection attacks
- Protect sensitive data by conducting privacy and risk assessments

Following these practices helps protect your web application from common attacks.

HTTP Version

The **HTTP version** shows the protocol version used to communicate between the client and server. Here's a quick rundown of the most common ones:

HTTP/0.9 (1991)The first version, only supported GET requests.

HTTP/1.0 (1996)Added headers and better support for different types of content, improving caching.

HTTP/1.1 (1997)Brought persistent connections, chunked transfer encoding, and better caching. It's still widely used today.

HTTP/2 (2015)Introduced features like multiplexing, header compression, and prioritisation for faster performance.

HTTP/3 (2022)Built on HTTP/2, but uses a new protocol (QUIC) for quicker and more secure connections.

Although HTTP/2 and HTTP/3 offer better speed and security, many systems still use **HTTP/1.1** because it's well-supported and works with most existing setups. However, upgrading to HTTP/2 or HTTP/3 can provide significant performance and security improvements as more systems adopt them.

Request Headers

Request Headers allow extra information to be conveyed to the web server about the request. Some common headers are as follows:

Common Request Headers

Request Header	Example	Description
Host	`Host: tryhackme.com`	Specifies the name of the web server the request is for.
User-Agent	`User-Agent: Mozilla/5.0`	Shares information about the web browser the request is coming from.
Referer	`Referer: https://www.google.com/`	Indicates the URL from which the request came from.
Cookie	`Cookie: user_type=student; room=introtowebapplication; room_status=in_progress`	Information the web server previously asked the web browser to store is held in cookies.
Content-Type	`Content-Type: application/json`	Describes what type or format of data is in the request.

Request Body

In HTTP requests such as POST and PUT, where data is sent to the web server as opposed to requested from the web server, the data is located inside the HTTP Request Body. The formatting of the data can take many forms, but some common ones are `URL Encoded`, `Form Data`, `JSON`, or `XML`.

- **URL Encoded (application/x-www-form-urlencoded)**A format where data is structured in pairs of key and value where (`key=value`). Multiple pairs are separated by an (`&`) symbol, such as `key1=value1&key2=value2`. Special characters are percent-encoded.***Example***

```
POST /profile HTTP/1.1Host: tryhackme.comUser-Agent: Mozilla/5.0Content-Type: appli
cation/x-www-form-urlencodedContent-Length: 33
```

```
name=Aleksandra&age=27&country=US
```

- **Form Data (multipart/form-data)**Allows multiple data blocks to be sent where each block is separated by a boundary string. The boundary string is the defined header of the request itself. This type of formatting can be used to send binary data, such as when uploading files or images to a web server.***Example***

```
POST /upload HTTP/1.1Host: tryhackme.comUser-Agent: Mozilla/5.0Content-Type: multip
art/form-data; boundary=----WebKitFormBoundary7MA4YWxkTrZu0gW

----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="username"

aleksandra
----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="profile_pic"; filename="aleksandra.jpg"Conten
t-Type: image/jpeg

[Binary Data Here representing the image]
----WebKitFormBoundary7MA4YWxkTrZu0gW--
```

- **JSON (application/json)**In this format, the data can be sent using the JSON (JavaScript Object Notation) structure. Data is formatted in pairs of name : value. Multiple pairs are separated by commas, all contained within curly braces { }.***Example***

```
POST /api/user HTTP/1.1Host: tryhackme.comUser-Agent: Mozilla/5.0Content-Type: appl
ication/jsonContent-Length: 62
{
    "name": "Aleksandra",
    "age": 27,
    "country": "US"
}
```

- **XML (application/xml)**In XML formatting, data is structured inside labels called tags, which have an opening and closing. These labels can be nested within each other. You can see in the example below the opening and closing of the tags to send details about a user called Aleksandra.***Example***

```
POST /api/user HTTP/1.1Host: tryhackme.comUser-Agent: Mozilla/5.0Content-Type: appl
ication/xmlContent-Length: 124
<user><name>Aleksandra</name><age>27</age><country>US</country></user>
```

When you interact with a web application, the server sends back an **HTTP response** to let you know whether your request was successful or something went wrong. These responses include a **status code** and a short explanation (called the **Reason Phrase**) that gives insight into how the server handled your request.

Status Line

The first line in every HTTP response is called the **Status Line**. It gives you three key pieces of info:

1. **HTTP Version**: This tells you which version of HTTP is being used.
2. **Status Code**: A three-digit number showing the outcome of your request.
3. **Reason Phrase**: A short message explaining the status code in human-readable terms.

Since we already covered HTTP Versions in Task 5, let's focus on the **Status Codes** and **Reason Phrases** here.

Status Codes and Reason Phrases

The **Status Code** is the number that tells you if the request succeeded or failed, while the **Reason Phrase** explains what happened. These codes fall into five main categories:

Informational Responses (100-199)

These codes mean the server has received part of the request and is waiting for the rest. It's a "keep going" signal.

Successful Responses (200-299)

These codes mean everything worked as expected. The server processed the request and sent back the requested data.

Redirection Messages (300-399)

These codes tell you that the resource you requested has moved to a different location, usually providing the new URL.

Client Error Responses (400-499)

These codes indicate a problem with the request. Maybe the URL is wrong, or you're missing some required info, like authentication.

Server Error Responses (500-599)

These codes mean the server encountered an error while trying to fulfil the request. These are usually server-side issues and not the client's fault.

Common Status Codes

Here are some of the most frequently seen status codes:

100 (Continue)

The server got the first part of the request and is ready for the rest.

200 (OK)

The request was successful, and the server is sending back the requested resource.

301 (Moved Permanently)

The resource you're requesting has been permanently moved to a new URL. Use the new URL from now on.

404 (Not Found)

The server couldn't find the resource at the given URL. Double-check that you've got the right address.

500 (Internal Server Error)

Something went wrong on the server's end, and it couldn't process your request.

Response Headers

When a web server responds to an HTTP request, it includes **HTTP response headers**, which are basically key-value pairs. These headers provide important info about the response and tell the client (usually the browser) how to handle it.

Picture an example of an HTTP response with the headers highlighted. Key headers like `Content-Type`, `Content-Length`, and `Date` give us important details about the response the server sends back.

Required Response Headers

Some response headers are crucial for making sure the HTTP response works properly. They provide essential info that both the client and server need to process everything correctly. Here are a few important ones:

- **Date**:

  Example: `Date: Fri, 23 Aug 2024 10:43:21 GMT`

  This header shows the exact date and time when the response was generated by the server.

- **Content-Type**:

  Example: `Content-Type: text/html; charset=utf-8`

  It tells the client what kind of content it's getting, like whether it's HTML, JSON, or something else. It also includes the character set (like UTF-8) to help the browser display it properly.

- **Server**:

  Example: `Server: nginx`

  This header shows what kind of server software is handling the request. It's good for debugging, but it can also reveal server information that might be useful for attackers, so many people remove or obscure this one.

Other Common Response Headers

Besides the essential ones, there are other common headers that give additional instructions to the client or browser and help control how the response should be handled.

- **Set-Cookie**:

  Example: `Set-Cookie: sessionId=38af1337es7a8`

  This one sends cookies from the server to the client, which the client then stores and sends back with future requests. To keep things secure, make sure cookies are set with the `HttpOnly` flag (so they can't be accessed by JavaScript) and the `Secure` flag (so they're only sent over HTTPS).

- **Cache-Control**:

  Example: `Cache-Control: max-age=600`

  This header tells the client how long it can cache the response before checking with the server again. It can also prevent sensitive info from being cached if needed (using `no-cache`).

- **Location**:

  Example: `Location: /index.html`

This one's used in redirection (3xx) responses. It tells the client where to go next if the resource has moved. If users can modify this header during requests, be careful to validate and sanitise it—otherwise, you could end up with open redirect vulnerabilities, where attackers can redirect users to harmful sites.

Response Body

The **HTTP response body** is where the actual data lives—things like HTML, JSON, images, etc., that the server sends back to the client. To prevent **injection attacks** like Cross-Site Scripting (XSS), always sanitise and escape any data (especially user-generated content) before including it in the response.

Security Headers

HTTP Security Headers help improve the overall security of the web application by providing mitigations against attacks like Cross-Site Scripting (XSS), clickjacking, and others. We will now dig deeper into the following security headers:

- Content-Security-Policy (CSP)
- Strict-Transport-Security (HSTS)
- X-Content-Type-Options
- Referrer-Policy

You can use a site like https://securityheaders.io/ to analyse the security headers of any website. After the discussion in this task, you will hopefully have a better understanding of what it is reporting on.

Content-Security-Policy (CSP)

A CSP header is an additional security layer that can help mitigate against common attacks like Cross-Site Scripting (XSS). Malicious code could be hosted on a separate website or domain and injected into the vulnerable website. A CSP provides a way for administrators to say what domains or sources are considered safe and provides a layer of mitigation to such attacks.

Within the header itself, you may see properties such as `default-src` or `script-src` defined and many more. Each of these give an option to an administrator to define at various levels of granularity, what domains are allowed for what type of content. The use of self is a special keyword that reflects the same domain on which the website is hosted.

Looking at an example CSP header:

`Content-Security-Policy: default-src 'self'; script-src 'self' https://cdn.tryhackme.com; style-src 'self'`

We see the use of:

- **default-src**which specifies the default policy of self, which means only the current website.
- **script-src**which specifics the policy for where scripts can be loaded from, which is self along with scripts hosted on `https://cdn.tryhackme.com`
- **style-src**which specifies the policy for where style CSS style sheets can be loaded from the current website (self)

Strict-Transport-Security (HSTS)

The HSTS header ensures that web browsers will always connect over HTTPS. Let's look at an example of HSTS:

```
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
```

Here's a breakdown of the example HSTS header by directive:

- **max-age**This is the expiry time in seconds for this setting
- **includeSubDomains**An optional setting that instructs the browser to also apply this setting to all subdomains.
- **preload**This optional setting allows the website to be included in preload lists. Browsers can use preload lists to enforce HSTS before even having their first visit to a website.

X-Content-Type-Options

The X-Content-Type-Options header can be used to instruct browsers not to guess the MIME time of a resource but only use the Content-Type header. Here's an example:

`X-Content-Type-Options: nosniff`

Here's a breakdown of the X-Content-Type-Options header by directives:

- **nosniff**This directive instructs the browser not to sniff or guess the MIME type.

Referrer-Policy

This header controls the amount of information sent to the destination web server when a user is redirected from the source web server, such as when they click a hyperlink. The header is available to allow a web administrator to control what information is shared. Here are some examples of Referrer-Policy:

- `Referrer-Policy: no-referrer`
- `Referrer-Policy: same-origin`
- `Referrer-Policy: strict-origin`
- `Referrer-Policy: strict-origin-when-cross-origin`

Here's a breakdown of the Referrer-Policy header by directives:

- **no-referrer**This completely disables any information being sent about the referrer
- **same-origin**This policy will only send referrer information when the destination is part of the same origin. This is helpful when you want referrer information passed when hyperlinks are within the same website but not outside to external websites.
- **strict-origin**This policy only sends the referrer as the origin when the protocol stays the same. So, a referrer is sent when an HTTPS connection goes to another HTTPS connection.
- **strict-origin-when-cross-origin**This is similar to strict-origin except for same-origin requests, where it sends the full URL path in the origin header.